From Separation Logic to Rust

Ownership as a type system

Cristina David

Lecture 13

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Five lectures of machinery

- Hoare logic: prove that a program obeys its specification (has no bugs).
- Separation logic: prove the same for programs that use pointers.
- Symbolic execution: find inputs that break a program (real bugs).

Five lectures of machinery

- Hoare logic: prove that a program obeys its specification (has no bugs).
- Separation logic: prove the same for programs that use pointers.
- Symbolic execution: find inputs that break a program (real bugs).

All of it costs effort: annotations, invariants, proofs. Someone must write them.

What if the language did it for you?

The question for today

We built a discipline for reasoning about aliased, mutable memory.

The question for today

We built a discipline for reasoning about aliased, mutable memory.

What if . . .

. . . a language baked that discipline into its type system, so the checking happens at compile time, with no additional specifications/proofs to write?

The question for today

We built a discipline for reasoning about aliased, mutable memory.

What if . . .

. . . a language baked that discipline into its type system, so the checking happens at compile time, with no additional specifications/proofs to write?

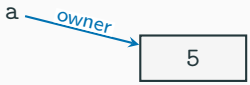
That language exists: **Rust**.

Rust in three ideas

(1) Ownership

Every value has exactly one owner.

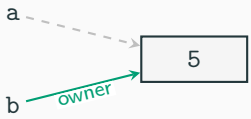
```
let a = Box::new(5); // on the heap
```



(1) Ownership

Every value has exactly one owner.

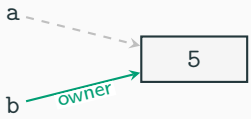
```
let a = Box::new(5); // on the heap
let b = a;           // move
```



(1) Ownership

Every value has exactly one owner.

```
let a = Box::new(5); // on the heap
let b = a;       // move
println!("{a}");
```

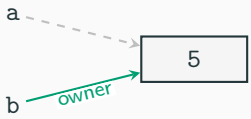


Is the last line accepted?

(1) Ownership

Every value has exactly one owner.

```
let a = Box::new(5); // on the heap
let b = a;           // move
println!("{a}");
```



Is the last line accepted?

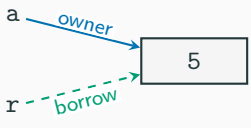
× No: a no longer owns the box.

When the owner goes out of scope, the value is dropped: if it lives on the heap, it is freed. If both owned it, both would free it: double free.

(2) Borrowing

To *use* a value without taking it, borrow it with `&`.

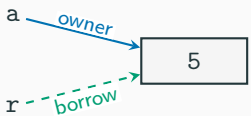
```
let a = 5;
let r = &a;          // borrow
println!("{a} {r}");   // fine ✓
```



(2) Borrowing

To *use* a value without taking it, borrow it with `&`.

```
let a = 5;
let r = &a;          // borrow
println!("{a} {r}");   // fine ✓
```



A borrow is temporary: when it ends, full access returns to the owner.

Owner vs borrower

	written	read	write	frees it	how many
owner	`T`	✓	✓*	✓	exactly one

Owner vs borrower

	written	read	write	frees it	how many
owner	`T`	✓	✓*	✓	exactly one
shared borrow	`&T`	✓	×	×	many at once

Owner vs borrower

	written	read	write	frees it	how many
owner	`T`	✓	✓*	✓	exactly one
shared borrow	`&T`	✓	×	×	many at once
unique borrow	`&mut T`	✓	✓	×	only one

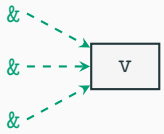
Owner vs borrower

	written	read	write	frees it	how many
owner	`T`	✓	✓*	✓	exactly one
shared borrow	`&T`	✓	×	×	many at once
unique borrow	`&mut T`	✓	✓	×	only one

Only the owner frees. A borrower uses the value; the owner gets it back.

* if declared `let mut x`; a plain `let x` guards against accidental writes.

(3) The rule



many & ✓

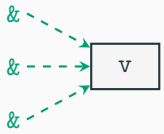
(3) The rule



many & ✓

one &mut ✓

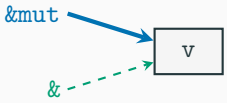
(3) The rule



many & ✓

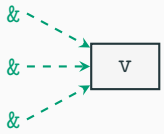


one &mut ✓



both ×

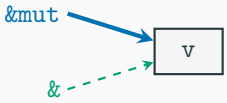
(3) The rule



many & ✓



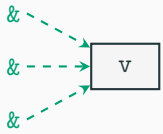
one `&mut` ✓



both ×

What exactly does the rule forbid?

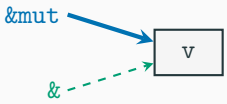
(3) The rule



many & ✓



one `&mut` ✓



both ×

What exactly does the rule forbid?

Aliasing together with mutation: what broke the assignment axiom in Lecture 10.

Ownership is separation logic in the type system

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value
$\{x \mapsto v\}\ f(x)\ \{\text{emp}\}$: f keeps the cell	in a call of `f`, `x` owns the value

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value
$\{x \mapsto v\}\ f(x)\ \{\text{emp}\}$: f keeps the cell	in a call of `f`, `x` owns the value
$\{x \mapsto v\}\ f(x)\ \{x \mapsto _\}$: f gives it back	in a call of `f`, `x` borrows the value

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value
$\{x \mapsto v\}\ f(x)\ \{\text{emp}\}$: f keeps the cell	in a call of `f`, `x` owns the value
$\{x \mapsto v\}\ f(x)\ \{x \mapsto _\}$: f gives it back	in a call of `f`, `x` borrows the value
$P * Q$: separate cells	a `&mut` never shares its cell

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value
$\{x \mapsto v\}\ f(x)\ \{\text{emp}\}$: f keeps the cell	in a call of `f`, `x` owns the value
$\{x \mapsto v\}\ f(x)\ \{x \mapsto _\}$: f gives it back	in a call of `f`, `x` borrows the value
$P * Q$: separate cells	a `&mut` never shares its cell
the frame rule	a function signature (next slides)

Ownership is separation logic in the type system

Separation logic	Rust
$x \mapsto v$: may read, write, free the cell	`x` owns the value
$\{x \mapsto v\}\ f(x)\ \{\text{emp}\}$: f keeps the cell	in a call of `f`, `x` owns the value
$\{x \mapsto v\}\ f(x)\ \{x \mapsto _\}$: f gives it back	in a call of `f`, `x` borrows the value
$P * Q$: separate cells	a `&mut` never shares its cell
the frame rule	a function signature (next slides)

In separation logic, you prove these. In Rust, the compiler checks them.

A `&mut` never shares its cell

```
let mut s = 5;
let a = &s;
let b = &s;
println!("{a} {b}");
```

```
let mut s = 5;
let a = &mut s;
let b = &s;
*a += 1;     // a still live
```

A `&mut` never shares its cell

```
let mut s = 5;
let a = &s;
let b = &s;
println!("{a} {b}");
```

✓ Accepted: both only read.

```
let mut s = 5;
let a = &mut s;
let b = &s;
*a += 1;      // a still live
```

A `&mut` never shares its cell

```
let mut s = 5;
let a = &s;
let b = &s;
println!("{a} {b}");
```

✓ Accepted: both only read.

```
let mut s = 5;
let a = &mut s;
let b = &s;
*a += 1;      // a still live
```

× Rejected: `s` is already borrowed uniquely.

A `&mut` never shares its cell

```
let mut s = 5;
let a = &s;
let b = &s;
println!("{a} {b}");
```

✓ Accepted: both only read.

```
let mut s = 5;
let a = &mut s;
let b = &s;
*a += 1;      // a still live
```

× Rejected: `s` is already borrowed uniquely.

In Rust, a never shares its cell, so the corresponding separation logic assertion is:

$$(a \mapsto _) * (b \mapsto _) \overset{a=b=s}{=\!=} (s \mapsto _) * (s \mapsto _) \equiv \text{false}$$

A `&mut` never shares its cell

```
let mut s = 5;
let a = &s;
let b = &s;
println!("{a} {b}");
```

✓ Accepted: both only read.

```
let mut s = 5;
let a = &mut s;
let b = &s;
*a += 1;      // a still live
```

× Rejected: `s` is already borrowed uniquely.

In Rust, a never shares its cell, so the corresponding separation logic assertion is:

$$(a \mapsto _) * (b \mapsto _) \overset{a=b=s}{=\!=} (s \mapsto _) * (s \mapsto _) \equiv \text{false}$$

The type checker rejects the overlap.

A signature is a frame rule

C vs Rust: the signature carries $*$

C

```
void f(int *x, int *y) {
  *x = 1; *y = 2;
}
```

Rust

```
fn f(x:  &mut i32, y:  &mut i32) {
  *x = 1; *y = 2;
}
```

C vs Rust: the signature carries $*$

C

```
void f(int *x, int *y) {
  *x = 1; *y = 2;
}
```

Rust

```
fn f(x:  &mut i32, y:  &mut i32) {
  *x = 1; *y = 2;
}
```

After `f`, is `*x == 1`?

C vs Rust: the signature carries $*$

C

```
void f(int *x, int *y) {
  *x = 1; *y = 2;
}
```

Rust

```
fn f(x: &mut i32, y: &mut i32) {
  *x = 1; *y = 2;
}
```

After `f`, is `*x == 1`?

Only if $x \neq y$. You write it, and prove it at every call:

$\{x \mapsto _ * y \mapsto _\}$ `f(x,y)` $\{x \mapsto 1 * y \mapsto 2\}$

✓ Always. Two `&mut` are disjoint by the type rules: the signature is the $*$.

C vs Rust: the signature carries $*$

C

```
void f(int *x, int *y) {
  *x = 1; *y = 2;
}
```

Rust

```
fn f(x: &mut i32, y: &mut i32) {
  *x = 1; *y = 2;
}
```

After `f`, is `*x == 1`?

Only if $x \neq y$. You write it, and prove it at every call:

$\{x \mapsto _ * y \mapsto _\}$ `f(x,y)` $\{x \mapsto 1 * y \mapsto 2\}$

```
f(&a, &a);  // compiles:  *x == 2 ×
```

✓ Always. Two `&mut` are disjoint by the type rules: the signature is the $*$.

```
f(&mut a, &mut a);  // rejected
```

C vs Rust: the signature is the frame

C

```
void incr(int *x) { *x += 1; }
int n = 1;
int *a = &n;
int *b = &n;       // alias
incr(a);
```

Rust

```
fn incr(x:  &mut i32) { *x += 1; }
let mut n = 1;
let a = &mut n;
let b = &mut n;  // alias
incr(a);
```

C vs Rust: the signature is the frame

C

```
void incr(int *x) { *x += 1; }
int n = 1;
int *a = &n;
int *b = &n;       // alias
incr(a);
```

Rust

```
fn incr(x:  &mut i32) { *x += 1; }
let mut n = 1;
let a = &mut n;
let b = &mut n;  // alias
incr(a);
```

After `incr(a)`, is `*b == 1`?

C vs Rust: the signature is the frame

C

```
void incr(int *x) { *x += 1; }
int n = 1;
int *a = &n;
int *b = &n;       // alias
incr(a);
```

Rust

```
fn incr(x:  &mut i32) { *x += 1; }
let mut n = 1;
let a = &mut n;
let b = &mut n;  // alias
incr(a);
```

After `incr(a)`, is `*b == 1`?

× No: `*b == 2`. `b` aliases `a`.

× Rejected by the compiler: two `&mut` to `n`.

C vs Rust: the signature is the frame

C

```
void incr(int *x) { *x += 1; }
int n = 1;
int *a = &n;
int *b = &n;       // alias
incr(a);
```

Rust

```
fn incr(x:  &mut i32) { *x += 1; }
let mut n = 1;
let a = &mut n;
let b = &mut n;  // alias
incr(a);
```

After `incr(a)`, is `*b == 1`?

× No: `*b == 2`. `b` aliases `a`.

× Rejected by the compiler: two `&mut` to `n`.

$$\frac{\{a \mapsto _\}\ \texttt{incr(a)}\ \{a \mapsto _\}}{\{a \mapsto _ * b \mapsto 1\}\ \texttt{incr(a)}\ \{a \mapsto _ * b \mapsto 1\}}\ \text{Frame}$$

C vs Rust: the signature is the frame

C

```
void incr(int *x) { *x += 1; }
int n = 1;
int *a = &n;
int *b = &n;       // alias
incr(a);
```

Rust

```
fn incr(x:  &mut i32) { *x += 1; }
let mut n = 1;
let a = &mut n;
let b = &mut n;  // alias
incr(a);
```

After `incr(a)`, is `*b == 1`?

× No: `*b == 2`. `b` aliases `a`.

× Rejected by the compiler: two `&mut` to `n`.

$$\frac{\{a \mapsto _\}\ \texttt{incr(a)}\ \{a \mapsto _\}}{\{a \mapsto _ * b \mapsto 1\}\ \texttt{incr(a)}\ \{a \mapsto _ * b \mapsto 1\}}\ \text{Frame}$$

C: you prove $a \neq b$, then Frame. Rust: the compiler frames.

Who supplies the footprint?

	Separation logic	Rust
footprint written as	a precondition	a *type*
frame rule applied by	you	the compiler
discharged by	a proof	type-checking

Who supplies the footprint?

	Separation logic	Rust
footprint written as	a precondition	a *type*
frame rule applied by	you	the compiler
discharged by	a proof	type-checking

Nothing comes for free

The borrow checker is automatic, fast, and needs no proofs.

So what did we give up?

Nothing comes for free

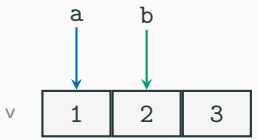
The borrow checker is automatic, fast, and needs no proofs.

So what did we give up?

Some *safe* programs are rejected.

Safe, but rejected

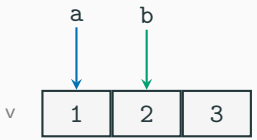
```
let mut v = vec![1, 2, 3];
let a = &mut v[0];
let b = &mut v[1];
*a += *b;
```



Two *different* elements: disjoint.

Safe, but rejected

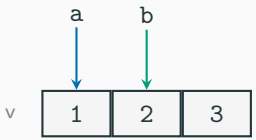
```
let mut v = vec![1, 2, 3];
let a = &mut v[0];
let b = &mut v[1];
*a += *b;
```



Two *different* elements: disjoint. Accepted?

Safe, but rejected

```
let mut v = vec![1, 2, 3];
let a = &mut v[0];
let b = &mut v[1];
*a += *b;
```

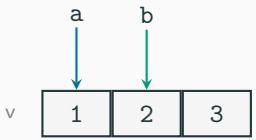


Two *different* elements: disjoint. Accepted?

× Rejected. The checker reasons about `v`, not about indices. Accepting needs $0 \neq 1$: in general, arbitrary arithmetic.

Safe, but rejected

```
let mut v = vec![1, 2, 3];
let a = &mut v[0];
let b = &mut v[1];
*a += *b;
```



Two *different* elements: disjoint. Accepted?

× Rejected. The checker reasons about `v`, not about indices. Accepting needs $0 \neq 1$: in general, arbitrary arithmetic.

That is a theorem prover, not a type checker.

Why some safe programs must be rejected

- Whether a program is memory-safe is undecidable.

Why some safe programs must be rejected

- Whether a program is memory-safe is undecidable.
- The borrow checker always terminates, with no annotations.

Why some safe programs must be rejected

- Whether a program is memory-safe is undecidable.
- The borrow checker always terminates, with no annotations.

So it must get some programs wrong.

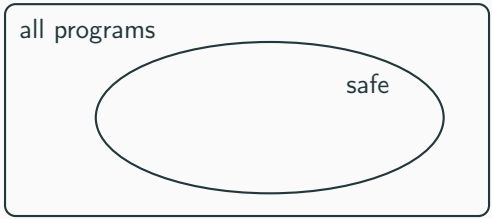
Why some safe programs must be rejected

- Whether a program is memory-safe is undecidable.
- The borrow checker always terminates, with no annotations.

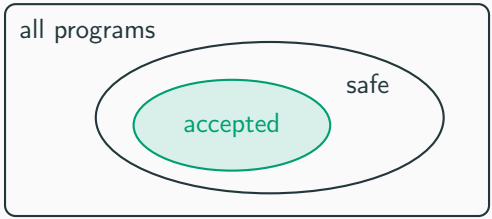
So it must get some programs wrong.

Rust's choice: never accept an unsafe program; sometimes reject a safe one.

Which way to be wrong?

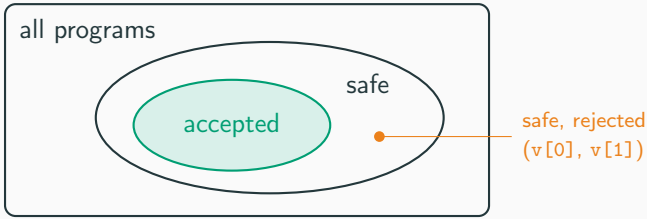


Which way to be wrong?



Every accepted program is safe: over-approximation, like Hoare logic.

Which way to be wrong?



Every accepted program is safe: over-approximation, like Hoare logic. Some safe programs fall outside: rejected.

The escape hatch

`unsafe`

`unsafe` Rust permits a few extra operations, e.g. dereferencing a raw pointer.

unsafe

`unsafe` Rust permits a few extra operations, e.g. dereferencing a raw pointer.

A raw pointer (`*mut T`, `*const T`) is a C pointer: no owner, no borrow tracking. It may be null, dangling, or aliased.

```
let p = &mut x as *mut i32;     // creating one:  safe
unsafe { *p = 1; }              // using one:  needs unsafe
```

`unsafe` Rust permits a few extra operations, e.g. dereferencing a raw pointer.

A raw pointer (`*mut T`, `*const T`) is a C pointer: no owner, no borrow tracking. It may be null, dangling, or aliased.

```
let p = &mut x as *mut i32;     // creating one:  safe
unsafe { *p = 1; }              // using one:  needs unsafe
```

What `unsafe` means

For `*p`, you must prove `p` points to live memory and is not wrongly aliased. The proof obligation moves from the compiler to you.

Undefined behaviour

Break a rule, e.g. allow aliasing together with mutation (two live `&mut` to one cell), and the language gives your program no meaning at all.

Undefined behaviour

Break a rule, e.g. allow aliasing together with mutation (two live `&mut` to one cell), and the language gives your program no meaning at all.

Nothing is obliged to tell you:

- it may work today, on this compiler;
- it may break with optimisations, or a new compiler;

Undefined behaviour

Break a rule, e.g. allow aliasing together with mutation (two live `&mut` to one cell), and the language gives your program no meaning at all.

Nothing is obliged to tell you:

- it may work today, on this compiler;
- it may break with optimisations, or a new compiler;

Safe Rust cannot cause UB. `unsafe` can.

Which one has undefined behaviour?

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```

Which one has undefined behaviour?

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

✓ Fine.

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```

× Undefined behaviour.

Which one has undefined behaviour?

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

✓ Fine.

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```

× Undefined behaviour.

Same lines, different order. So what tells them apart?

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

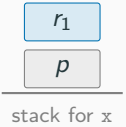


Push p.

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

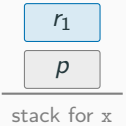


Make r_1 from p: push it.

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

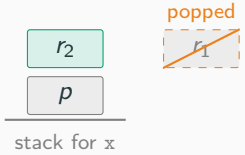


Use r_1: it is in the stack ✓

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```

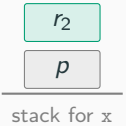


Make r_2 from p: pop everything above p, push r_2.

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```



Use r_2: in the stack ✓

Stacked Borrows, by example

Each location keeps a stack of the pointers allowed to use it (Jung et al., POPL 2020).

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
*r1 = 1;
let r2 = unsafe { &mut *p };
*r2 = 2;
```



Use r_2: in the stack ✓

Fine: r_1 was evicted, but never used again.

The other order

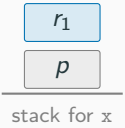
```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Push p.

The other order

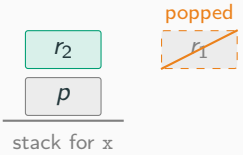
```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Make r_1 from p: push it.

The other order

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Make r_2 from p: pop r_1, push r_2.

The other order

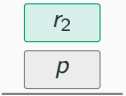
```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Use r_2: in the stack ✓

The other order

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Use r_1: not in the stack. $\times$ Undefined behaviour.

The other order

```
let mut x = 42;
let p = &mut x as *mut i32;
let r1 = unsafe { &mut *p };
let r2 = unsafe { &mut *p };
*r2 = 2;
*r1 = 1;
```



Use r_1: not in the stack. × Undefined behaviour.

The two rules

Make a reference from p: pop everything above p, push the new tag.

Use a tag: it must be in the stack; otherwise, undefined behaviour.

(Simplified: the real model distinguishes unique from shared tags, and reads from writes.)

Miri

Miri is an interpreter for Rust, from the Rust project: it runs your program step by step and checks each memory access, including `unsafe` code (`cargo miri run`).

The Stacked Borrows rules are operational, so Miri can run them, keeping a stack per location:

```
error: Undefined Behavior: attempting a write access
  using <t1> but that tag does not exist in the
  borrow stack for this location
```

Miri

Miri is an interpreter for Rust, from the Rust project: it runs your program step by step and checks each memory access, including `unsafe` code (`cargo miri run`).

The Stacked Borrows rules are operational, so Miri can run them, keeping a stack per location:

```
error: Undefined Behavior: attempting a write access
  using <t1> but that tag does not exist in the
  borrow stack for this location
```

It points at `*r1 = 1`, exactly where the trace said.

What does Rust actually *prove*?

Progress and preservation: the same shape as in the first half of this unit.

What does Rust actually *prove*?

Progress and preservation: the same shape as in the first half of this unit.

Type soundness ensures memory safety and data-race freedom: no use-after-free, no dangling pointers, no double free, no data races.

What does Rust actually *prove*?

Progress and preservation: the same shape as in the first half of this unit.

Type soundness ensures memory safety and data-race freedom: no use-after-free, no dangling pointers, no double free, no data races.

What it does *not* prove

Functional correctness.

What does Rust actually *prove*?

Progress and preservation: the same shape as in the first half of this unit.

Type soundness ensures memory safety and data-race freedom: no use-after-free, no dangling pointers, no double free, no data races.

What it does *not* prove

Functional correctness.

For that, we are back to Hoare logic.

Verifying Rust in practice

tool	how	what it checks
RustBelt	separation logic (Iris, Rocq)	the type system and `unsafe` libraries are sound
Miri	interpreter	no UB in one run, including `unsafe` code
Verus	SMT	functional correctness, against your specs
Prusti	separation logic (Viper)	no panics or overflows; your specs
Creusot	Why3	functional correctness
Kani	bounded model checking	assertions and no UB, up to a bound (AWS)

Verifying Rust in practice

tool	how	what it checks
RustBelt	separation logic (Iris, Rocq)	the type system and `unsafe` libraries are sound
Miri	interpreter	no UB in one run, including `unsafe` code
Verus	SMT	functional correctness, against your specs
Prusti	separation logic (Viper)	no panics or overflows; your specs
Creusot	Why3	functional correctness
Kani	bounded model checking	assertions and no UB, up to a bound (AWS)

Ownership handles the memory reasoning, so the prover only does the functional part.

Summing up

The whole arc

L8 Hoare logic

$\{P\}\ c\ \{Q\}$

say what it should do; prove it

L9 wp, VCs

$I \wedge b \Rightarrow \mathsf{wp}(c, I)$

a solver checks; you give I

L10 Sep. logic

$x \mapsto 3 * y \mapsto 5$

$*$ and Frame: reason locally

L11 Lists



list(i): proofs about real code

L12 Sym. exec.



fork, unroll: find real bugs

L13 Rust



the type system does it

The whole arc

L8 Hoare logic

$\{P\}\ c\ \{Q\}$

say what it should do; prove it

L9 wp, VCs

$I \wedge b \Rightarrow \mathrm{wp}(c, I)$

a solver checks; you give I

L10 Sep. logic

$x \mapsto 3 * y \mapsto 5$

$*$ and Frame: reason locally

L11 Lists



list(i): proofs about real code

L12 Sym. exec.



fork, unroll: find real bugs

L13 Rust



the type system does it

prove: over-approximate, you write annotations

find bugs: under-approx.

type-check: automatic, over-approx.

The whole arc

L8 Hoare logic	L9 wp, VCs	L10 Sep. logic	L11 Lists	L12 Sym. exec.	L13 Rust
$\{P\}\ c\ \{Q\}$	$I \wedge b \Rightarrow \mathsf{wp}(c, I)$	$x \mapsto 3 * y \mapsto 5$			
say what it should do; prove it	a solver checks; you give I	$*$ and Frame: reason locally	list(i): proofs about real code	fork, unroll: find real bugs	the type system does it

prove: over-approximate, you write annotations

find bugs: under-approx.

type-check: automatic, over-approx.

Different techniques for the same goal: reasoning about program state.

Like this? Do a PhD on it

Verification & static analysis

prove programs correct, find bugs without running them

Translating C to Rust

from unsafe legacy code to safe Rust

Correct code generation

LLM-written code, with formal guarantees

Like this? Do a PhD on it

Verification & static analysis

prove programs correct, find bugs without running them

Translating C to Rust

from unsafe legacy code to safe Rust

Correct code generation

LLM-written code, with formal guarantees

Interested in a PhD? Come and talk to me, or email me:

`cristina.david@bristol.ac.uk`

Bristol Programming Languages Research Group: `https://plrg-bristol.github.io/`

Questions?

ATiPL 2026 Minute Sheet